# CSC111 Winter 2026 Project 2: Charting Your Academic Path: A Graph-Based Course Planning Tool for UofT Students

Jack Anderson, Efren Medina Arias, Diego Jose Figarella Urbaneja, Tanish Ariyur

March 31, 2026

## Introduction

The University of Toronto offers hundreds of courses across the Faculty of Arts and Science, many of which sit behind long chains of prerequisite requirements. As students at UofT, planning our paths toward our degrees is often tedious and confusing. A student who wants to eventually enrol in an upper-year course has to work backwards through multiple layers of prerequisites, often discovering gaps in their plan only after the fact. This adds unnecessary stress and uncertainty to academic planning, regardless of what program or discipline a student is pursuing.

This problem maps naturally onto a directed graph, where each course is a vertex and each prerequisite relationship is a directed edge from the required course to the one it unlocks. UofT prerequisites are not simple enough to be stored as plain directed edges, though, since a prerequisite string like "(CSC148H1 AND CSC165H1) OR CSC240H1" is a logical condition, not a single course. Our implementation reflects this: `CourseGraph` stores courses as vertices, but instead of explicit edges, each vertex holds a `BooleanList` prerequisite tree that encodes which other courses unlock it. These `BooleanList` trees are the tree structure at the core of the project. The course codes appearing as leaves of a `BooleanList` implicitly define directed relationships between courses (what we call pseudo-edges), since each leaf represents a course that must be completed before the current one. The full graph of prerequisite-of relationships is therefore encoded in the collection of these trees, one per vertex, rather than in an explicit adjacency structure. When needed for visualization, `CourseGraph.to_networkx()` materializes these pseudo-edges into real directed edges by extracting all course codes from each vertex's `BooleanList` via `get_all_courses()` and adding an edge from each to the current course.

**Given a student's completed courses and a target course they wish to take, our program determines the prerequisite courses the student needs to complete to satisfy all requirements, guides them through selecting those courses interactively, and displays the prerequisite network in an interactive visual format so the student can see the full picture of what they are working toward.**

This problem has been studied in the context of academic advising systems, where course catalogues are typically modelled as directed acyclic graphs [2]. Our approach is consistent with that framing, with the added complexity of UofT's prerequisite strings, which mix AND and OR conditions and credit thresholds in a single natural-language string.

## Dataset Description

Our primary data source is the University of Toronto Faculty of Arts and Science 2025–2026 Academic Calendar [1]. We worked with an HTML version of the calendar, which contains for each course its course code, title, description, contact hours, breadth requirement, prerequisite requirements, and exclusion notices. A representative example from the calendar is:

```
CSC236H1 - Introduction to the Theory of Computation.
Prerequisite: (60% or higher in CSC111H1) and (60% or higher in
              MAT102H5 or MAT137Y1 or MAT157Y1)
```

We used the Python library `beautifulsoup4` [4] to parse the HTML calendar file and extract each course entry into a structured JSON file. Concretely, `parse.py` uses BeautifulSoup's `find` method to locate course entries within `class='view-content'`, then extracts the title, body, breadth, hours, prerequisite, and exclusion fields from each entry's sub-elements. The code and name are split from the title string, and the breadth number is parsed from the trailing parenthetical in that field. The result is written to `data/raw/courses.json`, a clean, reusable dataset that we can load without re-parsing the HTML each time.

The harder problem was converting the raw prerequisite strings into something the program can actually evaluate. A string like `"CSC148H1 and CSC165H1/ CSC240H1"` has to become a structured AND/OR tree. The more complex prerequisite strings (those with deep nesting, mixed conjunctions, or credit-threshold clauses) were post-processed using large language models (Claude and ChatGPT), an approach we received explicit approval for from our TA and professor. The final processed file used by the program at runtime is `data/courses_formatted_3.json`, where each entry's `prereq_tree` field encodes the full logical structure of its prerequisites as nested AND/OR nodes and credit-threshold nodes.

At runtime, `json_to_graph.py` reads this file and loads each entry's course code, name, hours, description, breadth number, `prereq_tree`, and exclusions into the graph. The raw prerequisite string is not kept; only the structured tree.

## Computational Overview

**How graphs and trees play a central role.** The entire program is built on top of `CourseGraph` in `course_graph.py`, a directed graph where each vertex is a `_CourseVertex` storing all course metadata. Every core operation (checking whether a student can take a course, finding what they should take next, enumerating all paths to a target, counting credits) is a query on this graph. There is no way to run the program without constructing and traversing it.

The connections between vertices are encoded as `BooleanList` prerequisite trees, one per vertex, defined in `boolean_list.py`. `BooleanList` is a fully custom-built recursive data structure, not backed by any external library, designed specifically to represent and evaluate the propositional logic of UofT prerequisite conditions. It is the core innovation of the project. Without it, none of the eligibility checking, path-finding, or interactive navigation is possible, because UofT's prerequisites cannot be reduced to a flat list of courses.

**Structure.** Each `BooleanList` node has an `operator` (either `'AND'` or `'OR'`) and an `items` list whose elements can be one of three types:

- A **course code string**: a leaf representing a single course that must be completed. Each such string is also a pseudo-edge in the graph: it encodes a directed prerequisite-of relationship from that course to the vertex holding the tree, without requiring an explicit entry in an adjacency structure.
- A **nested** `BooleanList`: a sub-condition with its own operator and items, allowing arbitrarily deep nesting. This is what lets the structure represent conditions like "(CSC148H1 AND CSC165H1) OR CSC240H1" faithfully: the outer node is an OR, and one of its children is an AND node containing the two courses.
- A `CreditCondition` node: a custom class representing a quantitative credit-threshold requirement such as "at least 1.0 credit in CSC." `CreditCondition` is also fully custom: `credits_satisfied()` infers credit weight directly from the course code suffix (`H` = 0.5, `Y` = 1.0) with no external lookup, and `calculate_credits_department()` filters by department prefix to handle requirements like "1.0 FCE in MAT."

**Evaluation.** `BooleanList.is_satisfied(completed)` is a recursive propositional logic evaluator: given a set of completed course codes, it traverses the tree and returns whether the entire formula is true. The dispatch to each item type is handled by the static method `evaluate_item(item, completed)`, which branches on `isinstance` checks. A string item is satisfied if it is in `completed`, a `CreditCondition` item delegates to `credits_satisfied()`, and a nested `BooleanList` item recurses back into `is_satisfied()`. AND nodes use Python's `all()` over their items, OR nodes use `any()`, both of which short-circuit. The result is a full propositional model checker for UofT prerequisite formulas, written from scratch using only the language features covered in CSC111.

This design is what makes a condition like the CSC236H1 prerequisite, *"(60% or higher in CSC111H1) and (60% or higher in MAT102H5 or MAT137Y1 or MAT157Y1)"*, evaluable in a single call to `is_satisfied()`. The outer AND node has two children: the first is a leaf for `CSC111H1`, and the second is an OR node containing `MAT102H5`, `MAT137Y1`, and `MAT157Y1`. No other standard data structure from the course (linked lists, plain trees, or flat dicts) could represent this without losing information about which combination of courses satisfies the requirement.

**Pseudo-edge extraction.** `BooleanList.get_all_courses()` recursively collects every course code string from any depth of the tree, skipping `CreditCondition` nodes since they do not name a specific course. This is used by `CourseGraph.to_networkx()`, which calls it on each vertex's `BooleanList` and adds a directed edge from every returned code to the current course, materializing the pseudo-edges into real edges in a `nx.DiGraph` for visualization. Outside of `to_networkx()`, no explicit edge set is ever constructed; all prerequisite relationships are accessed purely by evaluating the `BooleanList` trees in place.

**Major computations.**

1. *Eligibility checking.* `CourseGraph.is_eligible(code, completed)` returns whether a student who has completed the given set of courses can enrol in `code`. It returns `False` immediately if the student has already completed the course, `True` if the course has no prerequisites, and otherwise delegates to `BooleanList.is_satisfied(completed)`, which recursively evaluates each item via `evaluate_item()`. `eligible_courses(completed)` calls this across every vertex to return the full set of currently available courses.

2. *Next-course resolution.* `get_next_needed_courses(graph, target, completed)` in `algorithms.py` is the algorithm that drives the interactive interface. Given a target course the student cannot yet take, it walks the prerequisite tree to find the courses they should pick from right now to make progress. If the target is already eligible, it returns the target itself. If not, it passes the target's `BooleanList` to `_resolve_needed()`, which dispatches to `_resolve_and()` or `_resolve_or()` depending on the node type.

   For AND nodes, `_resolve_and()` collects suggestions from all unsatisfied children simultaneously, since the student has to complete all of them anyway. If a child course is not yet eligible itself, the method recurses into its own prerequisite tree to find what needs to happen first.

   OR nodes are the tricky part. When a prerequisite is "CSC165H1 or CSC240H1," the student who has already started taking CSC165H1-level courses should not suddenly be shown CSC240H1 as a new option. `_resolve_or()` handles this by first checking whether any branch is already partially satisfied via `_has_progress()`, which returns `True` if at least one sub-item in a branch is in `completed`. If there are branches with prior progress, `_partition_or_children()` separates them from the rest and `_resolve_or()` focuses only on those branches, ignoring the others entirely. This keeps the suggestions relevant rather than flooding the student with every possible alternative every time they take a course.

   A `visited` set is threaded through all recursive calls to prevent infinite loops when the prerequisite graph has cycles.

3. *Relevance filtering.* `get_relevant_courses(graph, target, completed)` computes the set of all courses that appear on any prerequisite path to `target`, stopping recursion at courses already in

`completed`. `eligible_relevant_courses()` intersects this with the eligible courses to return only courses that are both takeable now and actually useful for reaching the target.

4. *Credit counting.* `CourseGraph.credit_count(completed, department)` returns the total credits accumulated from a set of completed courses, with an optional filter by department prefix. Credit weight is inferred from the course code suffix (`H` = 0.5, `Y` = 1.0). This is useful for checking whether program-level credit requirements are met.

5. *Course search.* `search_courses(graph, query)` returns courses whose code or name contains the query string (case-insensitive), sorted so that exact code prefix matches come first. This backs the live search bars in the interface.

**Result reporting.** The program presents results through two interfaces.

The main interface is `CourseNavigator` in `interface.py`, a Tkinter application with a setup phase and a navigation phase. In the setup phase the student adds completed courses via a live search bar and types a target course. Clicking "Start navigating" switches to the navigation phase. The centre panel shows course cards generated by `get_next_needed_courses()`, sorted by department and level, with the target course highlighted in yellow if it is already reachable. Clicking a card adds that course to `completed` and `path`, then refreshes the options. The left panel tracks the path taken so far with a step counter and an undo button. The right panel shows course metadata on hover: credit weight, level, department, breadth category, a formatted prerequisite tree with checkmarks beside completed items, the exclusions list, and a clickable link to the course's page on the UofT calendar [1]. The session ends on a success screen when the target is added to `completed`, showing the full path taken and the total number of courses selected. `CourseNavigator` carries 18 instance attributes in total, one for each persistently referenced widget, so that every attribute is fully type-annotated in the class body. This exceeds `python_ta`'s default limit of 7; the `max-attributes` threshold was raised to 20 in the `python_ta` configuration for `interface.py` to accommodate the complete set of type annotations.

The visualization is `visualize_course_graph()` in `visualizations.py`. It renders the full course prerequisite graph as an interactive Plotly [3] figure. Node positions are computed by a custom hierarchical layout: `get_depths()` assigns each course a depth equal to the length of the longest prerequisite chain leading to it via topological sort (using NetworkX [5]), and `hierarchical_layout()` places courses at the same depth on the same vertical level. Two `go.Scatter()` traces are combined, one with `mode='lines'` for edges and one with `mode='markers+text'` for nodes, and rendered via `fig.show()`.

**New Python libraries.**

- `networkx` [5]: converts `CourseGraph` to a `nx.DiGraph` via `CourseGraph.to_networkx()`; used in `visualizations.py` for topological sorting and hierarchical depth assignment.
- `plotly` [3]: renders the interactive course graph in `visualizations.py` using `go.Scatter()` traces and `go.Figure()`.
- `beautifulsoup4` [4]: parses the HTML course calendar in `parse.py` to extract structured course data into JSON.

## Running Instructions

1. Install Python 3.13.5.
2. Install dependencies:

   ```
   pip install beautifulsoup4 lxml networkx plotly
   ```

3. Run the interactive interface:

```
python interface.py
```

The processed dataset (`data/courses_formatted_3.json`) is included with the submission, so no additional data download or preprocessing is required. The Tkinter window opens directly on launch.

To view the graph visualization instead, run `visualizations.py` directly (`python visualizations.py`), which opens an interactive Plotly figure in the browser.

## Changes to Project Plan

- **Graph and tree models both used.** The proposal described both a directed graph and a `CourseTree` and noted we were still figuring out which to use. The final implementation uses both. `CourseGraph` is the graph, with courses as vertices. The `BooleanList` on each vertex is the tree: a recursive AND/OR tree that encodes the prerequisite logic for that course and implicitly defines directed relationships (pseudo-edges) to the courses it lists as leaves. A separate `CourseTree` class as described in the proposal was not implemented; the tree structure lives inside the graph rather than alongside it. The student state (completed courses) is passed as a parameter to graph methods rather than being embedded in tree nodes, which avoids the node-explosion problem the proposal identified.

- **BooleanList and CreditCondition added.** The proposal described extracting course codes from prerequisite strings via regex and storing "multiple valid prerequisite combinations per vertex." This turned out to be insufficient. A flat list of combinations cannot distinguish "CSC148H1 AND (CSC165H1 OR CSC240H1)" from "(CSC148H1 AND CSC165H1) OR CSC240H1," since the grouping information is lost. The final implementation replaces this with `BooleanList`, a fully custom recursive AND/OR tree that preserves the full logical structure of each prerequisite condition and evaluates it directly against the student's completed set. `CreditCondition` was added alongside it to handle the subset of requirements expressed as credit thresholds rather than specific course codes. Neither class exists in any library; both are written from scratch specifically for this project.

- **Corequisites not implemented.** The proposal mentioned storing corequisites as a distinct edge type on the graph. In practice, UofT's calendar does not consistently separate corequisites from prerequisites in the HTML, so we did not implement a distinct corequisite edge type. Corequisite requirements that appeared in the prerequisite field were treated as regular prerequisites during parsing.

- **Command-line input replaced by Tkinter.** The proposal described a command-line interface for entering completed courses and target courses. The final implementation uses a Tkinter GUI with live search bars and clickable course cards, which is more usable and avoids requiring students to type exact course codes from memory.

- **Visualization colour scheme simplified.** The proposal planned to colour-code nodes as green (completed), yellow (on the recommended path), and red (target). The final visualization renders all nodes with the same colour since the full-graph view is not filtered to a student's specific path. The per-student colour-coding was moved into the Tkinter interface instead, where the target course card is highlighted in yellow and path items in the left panel are coloured green.

- **find_path_to → get_next_needed_courses.** The proposal described a `find_path_to(target)` method that returns all valid sequences to a target. The final implementation replaces this with `get_next_needed_courses()` in `algorithms.py`, which resolves the courses the student should pick from at each step interactively rather than computing all paths upfront. This fits the layer-by-layer navigation model of the interface better than a full path enumeration would.

- **get_credits → credit_count.** `get_credits(optional[Dept])` from the proposal became `CourseGraph.credit` with the same semantics.

## Discussion

The problem we set out to solve was one we ran into personally as UofT students, figuring out what courses you need to take before you can take the one you actually want. Working backwards through the calendar manually is tedious enough when prerequisites are simple, but UofT's prerequisite strings are often nested combinations of AND and OR conditions, sometimes with credit thresholds mixed in. The goal was to make that process something the program handles instead of the student.

The single most important design decision in the project was building `BooleanList`. Every other component (the graph, the navigation algorithm, the interface) depends on it. The insight behind it is that UofT prerequisite conditions are propositional logic formulas over course codes, and the right way to represent a logic formula is as a tree. Standard data structures from the course (linked lists, plain trees, flat dicts) cannot express "complete A AND B, OR just C" without collapsing the OR and the AND into something ambiguous. `BooleanList` solves this by being a fully recursive AND/OR tree where each node carries an operator and a list of items that can themselves be `BooleanList` nodes, giving it the ability to represent arbitrary nesting depth.

What makes `BooleanList` non-trivial to build is the three-way polymorphism of its items. A node's children can be course code strings, nested `BooleanList` nodes, or `CreditCondition` nodes, all handled uniformly by `evaluate_item()`, which dispatches on type. This means `is_satisfied()` never needs to know in advance what kind of item it is evaluating; it just calls `evaluate_item()` on each child and lets the dispatch handle the rest. Getting this right required thinking carefully about when recursion terminates (at string leaves and `CreditCondition` nodes, which do not recurse further) and making sure AND/OR short-circuiting via `all()` and `any()` did not skip evaluations that had side effects elsewhere in the tree.

`CreditCondition` is its own custom class rather than a special string format, which matters because credit requirements need to be evaluated against the student's accumulated credits, not against a set of course codes. `credits_satisfied()` infers whether a student meets the threshold by parsing credit weight directly from course code suffixes (`H` for 0.5, `Y` for 1.0), with an optional department filter, keeping the entire eligibility check self-contained.

The hardest part of getting `BooleanList` into the program was not writing the class itself but producing valid trees from the raw calendar data. The prerequisite strings from the HTML are formatted inconsistently: some use commas for AND, some use semicolons, some mix "or" and "/" in the same clause. The more complex strings (those with deep nesting, mixed conjunctions, and credit-threshold clauses in the same expression) were post-processed using large language models (Claude and ChatGPT) to produce valid `prereq_tree` dictionaries. We received explicit approval from our TA and professor to use this approach for the data processing step. Without a validated conversion, malformed trees would silently enter the graph and cause incorrect eligibility results across the entire dataset.

The `get_next_needed_courses()` algorithm in `algorithms.py` was the most interesting piece of code to write, and it is built entirely on top of `BooleanList`. It works by recursing through the prerequisite tree of the target course and asking, at each node, what does the student need to take right now to make progress? The AND case is straightforward (collect from all unsatisfied children at once), but OR nodes required more thought. The issue is that once a student has started taking courses down one branch of an OR prerequisite, you do not want to suggest the other branches anymore. If someone has already taken `CSC165H1` as part of getting to `CSC207H1`, showing them `CSC240H1` as an alternative at that point is not useful. `_has_progress()` and `_partition_or_children()` solve this by detecting whether any sub-item in a branch is already satisfied and, if so, restricting suggestions to those branches only. Getting this right took a few iterations; early versions would open up all branches simultaneously and produce suggestions that had nothing to do with where the student was in their path. The fix depended on being able to introspect the `BooleanList` tree structurally, asking not just "is this satisfied?" but "does any sub-item

show progress?", which is only possible because the tree is a custom data structure we built and control entirely.

One thing we did not get to is integrating exclusions into eligibility checking. The exclusion list is stored on every `_CourseVertex` and is displayed in the right panel of the interface, but `is_eligible()` currently only checks prerequisites, not exclusions. This means the program could in theory suggest a course that a student is technically excluded from because they took an equivalent. Fixing this is straightforward: `is_eligible()` would just need to return `False` if any entry in `vertex.exclusions` appears in `completed`, but we ran out of time to add it cleanly given the number of edge cases in how the exclusion strings are formatted in the raw data.

The Plotly visualization works well for smaller subsets of the graph, for example all CSC-prefixed courses or the prerequisite chain for a single target. At the full scale of roughly 1,300 courses it becomes too dense to read. A natural extension would be to filter the visualization to only the courses returned by `get_relevant_courses()`, which already computes exactly the subgraph relevant to a given target. Passing that directly to `visualize_course_graph()` would give the student a readable view of just the courses that matter for their goal, coloured by whether they have been completed, which was the original intent of the colour-coding scheme from the proposal.

Overall, the core of what we set out to build works: a student can open the program, type in a target like `CSC373H1`, add the courses they have already taken, and then click through the suggested course cards until the target appears as an option. The prerequisite tree panel in the right sidebar shows exactly where they are in the requirement chain, with checkmarks beside the parts they have already satisfied. That was the original goal, and it does that reliably.

## References


[1] University of Toronto Faculty of Arts & Science. "2025–2026 Academic Calendar." *University of Toronto*, 2025. Available at: `https://artsci.calendar.utoronto.ca/`. Accessed March 2026.

[2] Barachini, F., and Stary, C. "From Knowledge Representation to Knowledge Communication: Course Prerequisite Graphs in Academic Advising." *Proceedings of the 2nd International Conference on Computer Supported Education*, 2010, pp. 135–144.

[3] Plotly Technologies Inc. "Plotly Python Open Source Graphing Library." *Plotly*, 2025. Available at: `https://plotly.com/python/`. Accessed March 2026.

[3a] Plotly Technologies Inc. "Network Graphs in Python." *Plotly*, 2025. Available at: `https://plotly.com/python/network-graphs/`. Accessed March 2026.

[4] Richardson, L. "Beautiful Soup Documentation." *Crummy*, 2025. Available at: `https://www.crummy.com/software/BeautifulSoup/bs4/doc/`. Accessed March 2026.

[5] Hagberg, A., Swart, P., and Chult, D. S. "Exploring Network Structure, Dynamics, and Function using NetworkX." *Proceedings of the 7th Python in Science Conference*, 2008, pp. 11–15.